

Learning with AI — Topic 1

Python Computer Vision with OpenCV and MediaPipe for Real-Time Pose Estimation

Aaryans Nepal · CSC 494 — IoT (Spring 2026)

What I Wanted to Learn

Run real-time human **pose estimation** on a Raspberry Pi 5, on **CPU only** — no GPU, no cloud, no heavy framework.

Goal: get a skeleton out of every camera frame fast enough to react to posture changes in real life.

The Two Tools

- **OpenCV** — reads camera frames, does image rotation, draws overlays, encodes JPEGs for the web stream.
- **MediaPipe Pose (BlazePose)** — Google's pose-estimation model that outputs **33 body landmarks** per frame.

Together: `cv2.VideoCapture` → `frame` →
`mediapipe.Pose.process()` → 33 landmarks → your code.

How BlazePose Is Actually Fast

BlazePose is **two networks stitched together**:

1. **Detector** — scans the whole frame to find a person. Slow.
2. **Landmark tracker** — predicts 33 joints from a crop around the person. Fast.

After the first frame, the **detector is skipped** and the tracker uses the previous position. That's why FPS jumps on frame 2 and stays up.

On the Pi 5 CPU: **8-12 FPS**, no GPU.

What Each Landmark Gives You

For each of the 33 points:

- `x, y` — normalized 0-1 in the image
- `z` — depth relative to the hip midpoint
- `visibility` — 0-1 confidence

Practical rule I learned: always filter on `visibility > 0.5` before making a decision. A hand over the face produces garbage landmarks the moment visibility drops.

The Lesson the AI Did Not Give Me

I asked an AI to help me detect slouching using **X/Y** coordinates. It confidently helped. It didn't work — score stayed 0-2 out of 100 no matter how much I slouched.

I wrote a tiny **diagnostic script** that logged every metric's delta between "good" and "slouched" frames:

- `head_forward` (X-based): changed by **0.003**
- `nose_z_depth` : changed by **0.062**
- `face_size` : **doubled**

Slouching is **depth** (Z), not lateral (X/Y). The AI never suggested this because I had never asked.

The Meta-Lesson

AI will help you build a confident, broken system if you give it a confident, broken hypothesis.

It fills in your gaps; it does not challenge your frame.

What worked: stop asking *"help me implement X"*. Start asking *"here's data — what does the data say?"*

The diagnostic script changed the project. The AI could not have produced it alone, because the AI was not the one confused.

Takeaways

- **OpenCV + MediaPipe** is a legitimate production stack on a Pi 5 CPU.
- Landmarks come with `visibility` — always gate on it.
- BlazePose is fast because of architecture (detector + tracker), not magic.
- When your AI-assisted code doesn't work, **write a diagnostic**. Let the data push back.

Thank You

Source: https://github.com/AaryansNepal/CSC-494_Learning-with-AI